

Architecture Specification

Lab 3: Component Modelling & Architectural Pattern Selection

System: Vaccination Cohort & Dose Scheduling System (Problem Statement #17)

Prepared by: Shrihari Nagesh, PES1UG24CS445

1. Scenario review

1.1 What the system must do

ID	Requirement (short)
FR-001	Block Dose 2 booking until 28 days after Dose 1
FR-002	Register a citizen and assign a cohort from age, comorbidity and occupation
FR-003	Search centres by pincode and date, book a slot within capacity
FR-004	Officer checks in a citizen and records the dose with a batch number
FR-005	Issue a signed QR certificate after the final dose
FR-006	Verify a certificate by scanning its QR code
FR-007	Send a booking confirmation and a reminder

1.2 How well it must do it

ID	Requirement (short)
NFR-001	Certificate verification in under 150 ms
NFR-002	Data encrypted at rest and in transit, officers only
NFR-003	5,000 concurrent bookings, 95% answered within 3 s
NFR-004	Dose recording and verification keep working if booking is down
NFR-005	Officer ID and date on every dose record

1.3 Main challenges

Area	Challenge
Performance	Load is very uneven. Booking gets a sudden rush when a cohort opens, while the other functions stay at normal volume
Security	Health data is sensitive, and a forged certificate defeats the purpose of the system
Reliability	Vaccination at the centre must not stop because the public booking site is busy
Usability	Citizens may be first-time users, and officers work under time pressure

2. Architectural style analysis

Style	Strengths here	Weaknesses here	Fit
Layered	Simple to build and understand, clear separation of presentation, business and data	One deployable unit. A booking rush means scaling everything, and a fault in one layer affects every function	Medium
Client-Server	Central control, simple deployment, easy data consistency	The single server is a bottleneck and a single point of failure during a rollout	Low
Microservices	Each function scales and fails on its own, the signing key can be isolated in one service	More moving parts, network calls between services, data consistency across databases	High

3. Architecture selection

We chose Microservices Architecture for the Vaccination Cohort & Dose Scheduling System.

3.1 Two reasons

1. **The load is uneven, so parts must scale separately.** When a cohort opens, thousands of citizens book at once (NFR-003), while dose recording and certificate checks stay at normal volume. Only the Slot Booking Service needs extra instances for the rollout window.
2. **A failure in one area must not stop vaccination at the centre.** Booking, dose recording and certificate checks are separate services with separate databases. If booking is overloaded, officers can still record doses and verify certificates (NFR-004).

3.2 Security advantage

The private key that signs QR certificates lives only inside the Certificate Service. The public-facing services and their databases cannot reach it, so breaking into the booking service does not let an attacker forge a certificate. All outside traffic also enters through the API Gateway, which checks the login token and the role before any service is reached (NFR-002).

3.3 Performance benefit

Certificate verification is a small, stateless service, so it never waits behind booking traffic. It can stay within 150 ms (NFR-001) during a booking peak, while the booking service scales out on its own to hold its 3-second target.

3.4 Trade-off accepted

Microservices add network calls and deployment work. This design limits the cost: slot reservation stays inside one service and one database, so double booking is prevented by one local transaction. Only four interactions cross between services, and two of them are asynchronous.

4. Components and interfaces

4.1 Components

#	Component	Group	Responsibility	Requirements
1	Citizen Portal	Client	Register, book a slot, download the certificate	FR-002, FR-003, FR-005
2	Officer Console	Client	Check in, record a dose, scan a QR certificate	FR-004, FR-006
3	API Gateway	Edge	Single entry point: token check, role check, rate limiting, routing	NFR-002
4	Citizen Registry Service	Service	Profiles and cohort assignment	FR-002
5	Slot Booking Service	Service	Centres, slots, capacity, reservations	FR-003, NFR-003
6	Vaccination Record Service	Service	Dose records and dose-interval rules	FR-001, FR-004, NFR-005
7	Certificate Service	Service	Signs and verifies QR certificates	FR-005, FR-006, NFR-001
8	Notification Service	Service	Booking confirmations and reminders	FR-007

Data stores, one per service: Citizen DB, Booking DB, Records DB, Certificate Store.

4.2 Interfaces (ball and socket)

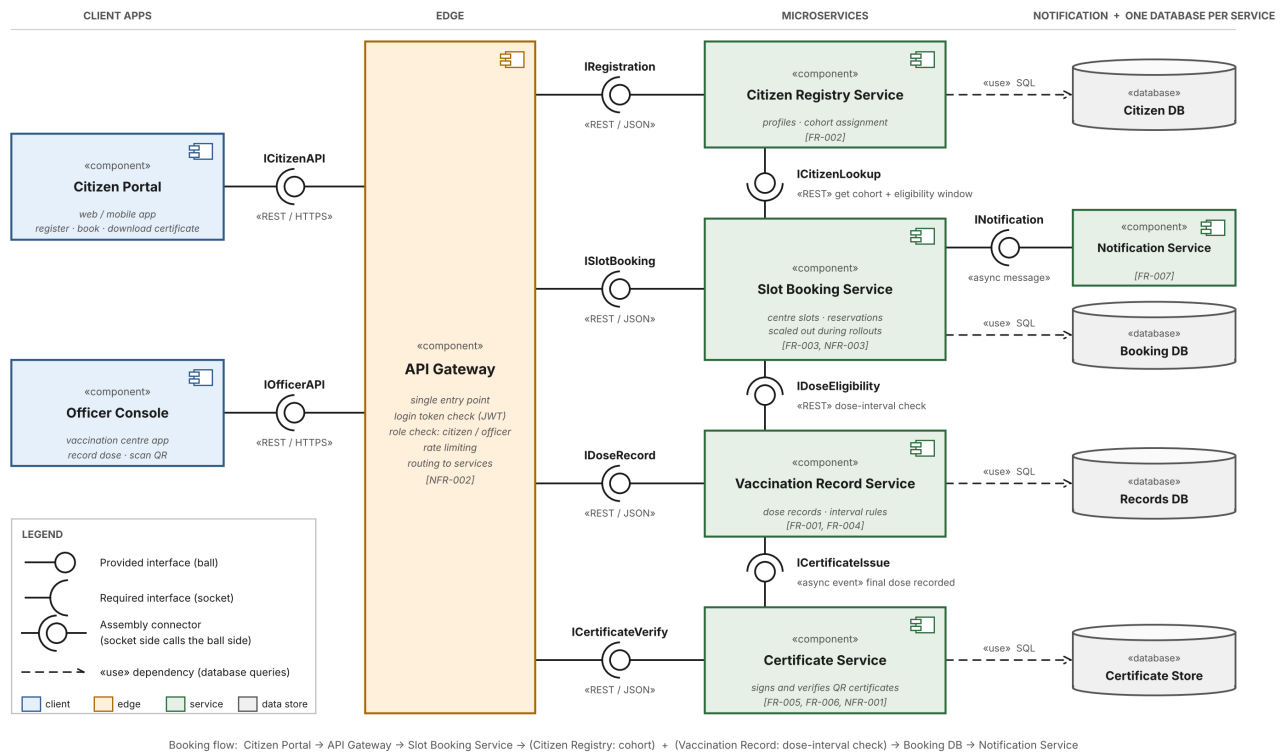
#	Interface	Provided by (ball)	Required by (socket)	Protocol	Main operations
1	ICitizenAPI	API Gateway	Citizen Portal	REST / HTTPS	register, search slots, book, get certificate
2	IOfficerAPI	API Gateway	Officer Console	REST / HTTPS	check in, record dose, verify certificate
3	IRegistration	Citizen Registry Service	API Gateway	REST / JSON	createProfile, updateProfile
4	ISlotBooking	Slot Booking Service	API Gateway	REST / JSON	searchSlots, bookSlot
5	IDoseRecord	Vaccination Record Service	API Gateway	REST / JSON	recordDose, getHistory
6	ICertificateVerify	Certificate Service	API Gateway	REST / JSON	verify(qrPayload)
7	ICitizenLookup	Citizen Registry Service	Slot Booking Service	REST	getCohort(citizenId)
8	IDoseEligibility	Vaccination Record Service	Slot Booking Service	REST	daysUntilEligible(citizenId, date)
9	ICertificateIssue	Certificate Service	Vaccination Record Service	Async event	finalDoseRecorded(citizenId)
10	INotification	Notification Service	Slot Booking Service	Async message	bookingConfirmed, reminderDue

Each service reaches its own database through a «use» dependency (SQL). No service reads another service's database.

5. UML component diagram

UML Component Diagram — Vaccination Cohort & Dose Scheduling System

Microservices Architecture · SE Lab 3 · Problem Statement #17 · Shrihari Nagesh (PES1UG24CS445)



Notation used:

- Rectangle with «component» and the component icon: a component
- Circle on a line (ball): an interface the component provides
- Half circle on a line (socket): an interface the component requires
- Ball inside socket: assembly connector. The socket side calls the ball side
- Dashed arrow with «use»: dependency on a data store

6. Data flow

Booking a slot

1. Citizen Portal calls **ICitizenAPI** on the API Gateway.
2. The gateway checks the token and role, then calls **ISlotBooking**.
3. Slot Booking Service calls **ICitizenLookup** to confirm the citizen's cohort is open.
4. Slot Booking Service calls **IDoseEligibility** to confirm the dose interval has passed.
5. It checks capacity and takes the seat in one transaction on the Booking DB.
6. It sends **bookingConfirmed** through **INotification**.

Recording a dose and issuing the certificate

1. Officer Console calls **IOfficerAPI**, and the gateway calls **IDoseRecord**.
2. Vaccination Record Service validates the batch number and saves the record in the Records DB.
3. If this was the final dose, it raises **finalDoseRecorded** through **ICertificateIssue**.
4. Certificate Service signs the certificate and stores it in the Certificate Store.

Verifying a certificate

1. Officer Console scans the QR code and calls `IOfficerAPI`.
2. The gateway calls `ICertificateVerify`.
3. Certificate Service checks the signature and answers Valid or Invalid.

7. Handout checklist

Handout item	Where
At least 5 components	8 components (section 4.1)
At least 4 interfaces	10 interfaces (section 4.2)
Provided and required interface notation	Ball and socket on every connector
Interfaces labelled with protocol	REST, async event, async message, SQL
Data flow and interactions	Section 6 and the footer of the diagram
Diagram as PNG or PDF	<code>Architecture_Diagram.png</code> , <code>Architecture_Diagram.pdf</code>
Justification, Word, one page, also PDF	<code>Architecture_Justification_Document.docx</code> , <code>.pdf</code>
Architecture choice, two reasons, security, performance	Section 3